МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РЕСПУБЛИКИ КАЗАХСТАН
ЭКОНОМИЧЕСКИЙ КОЛЛЕДЖ УНИВЕРСИТЕТА НАРХОЗ

КУРСОВАЯ РАБОТА

ТЕМА: Разработка системы учёта клиентов и заказов

ДИСЦИПЛИНА: _____

ВЫПОЛНИЛ
ФИО студента _____
Специальность _____
Курс _____
Группа _____

ПРОВЕРИЛ
ФИО руководителя _____
Оценка _____

АЛМАТЫ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ ... 3
ГЛАВА I. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ УЧЁТА КЛИЕНТОВ И ЗАКАЗОВ ... 5
1.1 Учёт клиентов и заказов на малом предприятии ... 5
1.2 Существующие способы учёта ... 6
1.3 Требования к разрабатываемой системе ... 7
1.4 Выбор технологий ... 8
ГЛАВА II. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ CLIENTFLOW ... 11
2.1 Архитектура приложения ... 11
2.2 Модель данных ... 12
2.3 Рабочие пространства и роли ... 13
2.4 Заказы и их жизненный цикл ... 14
2.5 Учёт оплат ... 16
2.6 Дополнительные возможности ... 17
2.7 Пользовательский интерфейс ... 19
ГЛАВА III. БЕЗОПАСНОСТЬ, ТЕСТИРОВАНИЕ И ВНЕДРЕНИЕ СИСТЕМЫ ... 21
3.1 Защита учётных записей и данных ... 21
3.2 Тестирование ... 22
3.3 Развёртывание ... 24
3.4 Проблемы при разработке и ограничения системы ... 24
3.5 Направления развития ... 25
ЗАКЛЮЧЕНИЕ ... 26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 27
ПРИЛОЖЕНИЯ ... 29

ВВЕДЕНИЕ

Небольшие компании сферы услуг, такие как мастерские, студии, ателье или частные исполнители, обычно начинают вести заказы в том, что есть под рукой. Контакты клиентов хранятся в телефоне, договорённости остаются в переписке в WhatsApp, а суммы и сроки записываются в тетрадь или в таблицу Excel. Пока заказов мало и работает один человек, этого хватает. Трудности начинаются, когда заказов становится несколько десятков в месяц и над ними работают два или три сотрудника. Кто-то пропускает срок, предоплата записана в одном месте, а доплата в другом, и уже нельзя быстро сказать, сколько клиент должен на самом деле. Эта ситуация и стала отправной точкой моей курсовой работы.

Предпринимательский кодекс Республики Казахстан выделяет субъектов малого предпринимательства в отдельную категорию по численности работников и годовому доходу [1]. У таких предприятий, как правило, нет своего программиста, а покупка и настройка крупной учётной системы для них не окупается. Готовые CRM-системы в основном рассчитаны на отделы продаж: в них есть воронки, сделки, интеграция с телефонией, и небольшой команде непросто разобраться, какая часть этих функций ей вообще нужна. Мне хотелось сделать инструмент, в котором есть только необходимое для ежедневной работы: клиенты, заказы со сроками и статусами, оплаты и понятный остаток долга.

Есть и правовая сторона вопроса. Имена и телефоны клиентов относятся к персональным данным, которые нужно защищать [2], а общая таблица, доступная каждому сотруднику, такой защиты не даёт.

Цель работы: разработать веб-приложение ClientFlow для учёта клиентов, заказов и оплат на малом предприятии с возможностью совместной работы нескольких сотрудников.

Для достижения цели были поставлены следующие задачи:

– рассмотреть, как малые предприятия обычно ведут учёт клиентов и заказов и какие при этом возникают проблемы;

– сравнить существующие способы учёта и сформулировать требования к системе;

– выбрать и обосновать технологии для серверной и клиентской частей приложения;

– спроектировать архитектуру приложения и структуру базы данных;

– реализовать учёт клиентов, заказов и оплат, роли сотрудников, уведомления и отчёты;

– защитить учётные записи пользователей и разделить данные разных организаций;

– проверить работу системы автоматическими тестами и подготовить её к развёртыванию.

Объект исследования: процесс учёта клиентов, заказов и расчётов с клиентами на малом предприятии сферы услуг.

Предмет исследования: программные средства автоматизации этого учёта на основе веб-технологий.

В работе использованы методы анализа и сравнения (при обзоре существующих решений и выборе технологий), проектирования реляционных баз данных, объектно-ориентированного программирования на языках Java и TypeScript, а также автоматического тестирования.

Практическая значимость работы состоит в том, что ClientFlow можно использовать в реальной работе небольшой команды. Приложение открывается в браузере на компьютере и на телефоне, поддерживает казахский, русский и английский языки и не требует установки. Для его размещения в интернете достаточно бесплатных тарифов облачных сервисов.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе рассмотрены особенности учёта клиентов и заказов в малом бизнесе, существующие решения и выбор технологий. Вторая глава посвящена проектированию и разработке приложения. В третьей главе описаны защита данных, тестирование, развёртывание, а также ограничения системы и возможные пути её развития.

ГЛАВА I. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ УЧЁТА КЛИЕНТОВ И ЗАКАЗОВ

1.1 Учёт клиентов и заказов на малом предприятии

Учёт клиентов и заказов отвечает на вопросы, которые владелец небольшого бизнеса задаёт себе почти каждый день. Кто наши клиенты и как с ними связаться? Какие заказы сейчас в работе и когда их нужно сдать? Кто из сотрудников за них отвечает? Сколько денег уже получено и сколько ещё должны заплатить? Если на эти вопросы нельзя ответить за минуту, предприятие теряет деньги, хотя со стороны это заметно не сразу.

В таком учёте можно выделить несколько основных объектов. Первый из них, клиент, описывается именем, телефоном, электронной почтой, иногда названием компании и адресом. Сюда же относятся заметки: что клиент предпочитает, о чём с ним договорились в прошлый раз. Второй объект, заказ, отвечает на вопрос «что нужно сделать». У заказа есть название, описание, сумма, валюта, срок выполнения, текущий статус и ответственный сотрудник. Третий объект, оплата, фиксирует получение денег: сумму, дату, способ (наличные, карта, перевод) и, при необходимости, номер чека или платёжного документа.

Связь между заказом и оплатами не всегда простая. В сфере услуг часто берут предоплату, а остаток получают после выполнения работы. Бывает, что клиент платит в три или четыре приёма, частью наличными, частью переводом. Иногда платёж нужно вернуть, например если заказ отменили. Чтобы в любой момент знать остаток долга, нужно учитывать все платежи по заказу, а ошибочные не удалять, а помечать как отменённые с указанием причины. Иначе через месяц уже никто не вспомнит, откуда взялась разница в сумме.

Программы, которые помогают вести клиентов и историю работы с ними, называют CRM-системами (от англ. Customer Relationship Management, управление взаимоотношениями с клиентами). В крупных компаниях CRM обычно связана с продажами: менеджер ведёт потенциального клиента по воронке от первого звонка до заключения сделки. Для мастерской или студии воронка продаж менее важна. Клиент чаще всего приходит сам, а основная работа начинается после того, как заказ принят. Поэтому в своём проекте я делаю упор не на продажи, а на исполнение заказа и расчёты по нему.

Когда учёт ведётся вручную, на практике повторяются одни и те же проблемы:

– один клиент оказывается записан дважды, с разными номерами телефона или с опечаткой в имени, и история его заказов разделяется на две части;

– срок выполнения записан, но никто не напоминает о нём заранее, и про заказ вспоминают, когда клиент уже звонит с претензией;

– остаток долга считают в уме или формулой в таблице, и после удаления одной строки формула начинает показывать неверную сумму;

– неизвестно, кто и когда изменил сумму заказа или отметил оплату;

– все сотрудники видят все данные, включая финансовые, хотя большинству из них нужны только их собственные заказы.

Последний пункт связан с защитой персональных данных. Закон Республики Казахстан «О персональных данных и их защите» относит к ним любые сведения, по которым можно определить конкретного человека, и обязывает собственника и оператора базы данных принимать правовые, организационные и технические меры для их защиты [2]. Закон «Об информатизации» дополнительно регулирует создание и использование информационных систем и устанавливает требования к защите электронных информационных ресурсов [3]. Для небольшой компании это означает как минимум две вещи: доступ к базе клиентов должен быть только у тех, кому он нужен по работе, и должно быть видно, кто что делал с данными.

1.2 Существующие способы учёта

Прежде чем проектировать собственное приложение, имеет смысл посмотреть, чем малые предприятия пользуются сейчас и чего им в этих инструментах не хватает.

Самый простой вариант, бумажная тетрадь или журнал заказов, ничего не стоит и не требует обучения. Однако в тетради нельзя быстро найти клиента по номеру телефона, её нельзя открыть одновременно двум сотрудникам, а если она потеряется, данные пропадут вместе с ней. Посчитать выручку за месяц можно только вручную.

Электронные таблицы Excel и Google Sheets заметно удобнее. Они знакомы почти всем, позволяют сортировать и фильтровать записи, а Google Sheets ещё и даёт нескольким людям работать с одним файлом. Слабое место таблицы в том, что она ничего не знает о правилах бизнеса. Ничто не мешает ввести отрицательную сумму, поставить оплату больше стоимости заказа или случайно стереть чужую строку. Ограничить права на уровне отдельных заказов сложно, поэтому обычно каждый сотрудник видит весь файл целиком. История версий в Google Sheets есть, но восстановить по ней, кто и почему поменял сумму конкретного заказа, неудобно.

Мессенджеры вроде WhatsApp и Telegram используются для общения с клиентами, и многие предприниматели фактически ведут заказы прямо в переписке. Это удобно в моменте, но информация оказывается разбросана по десяткам чатов, а при смене сотрудника или телефона часть истории теряется.

Готовые облачные CRM-системы, например Bitrix24 [4] и amoCRM [5], решают большинство перечисленных проблем. В них есть карточки клиентов, сделки, задачи, права доступа и отчёты. При этом они рассчитаны в первую очередь на продажи и на компании с отдельным отделом продаж, поэтому настройка под простой учёт заказов мастерской занимает время. Бесплатные тарифы у таких сервисов ограничены, а полный набор функций доступен по подписке. Отдельно можно упомянуть программы на платформе 1С, но они ориентированы на бухгалтерский и складской учёт и обычно требуют покупки лицензии и помощи специалиста при внедрении.

Сравнение этих вариантов по основным для малого бизнеса критериям приведено в таблице 1.

Таблица 1. Сравнение способов учёта клиентов и заказов

Критерий	**Тетрадь**	**Таблица Excel / Google Sheets**	**Облачная CRM**	**1С**
Стоимость начала работы	нет	нет	бесплатный тариф с ограничениями или подписка	лицензия и внедрение
Совместная работа	нет	в Google Sheets	да	да
Контроль сумм и остатка долга	вручную	формулами, без защиты от ошибок	да	да
Права доступа по ролям	нет	на уровне файла	да	да
Напоминания о сроках	нет	нет	да	зависит от конфигурации
Сложность освоения	низкая	низкая	средняя	высокая

По таблице видно, что простые инструменты бесплатны и понятны, но не защищают от ошибок и не разграничивают доступ. Профессиональные системы это умеют, но для команды из трёх человек они избыточны. Мне кажется, между этими крайностями есть место для приложения, которое осваивается за несколько минут, но при этом само следит за суммами, сроками и правами сотрудников. Создание такого приложения и стало целью моей работы.

1.3 Требования к разрабатываемой системе

Требования к ClientFlow я формулирую исходя из проблем, описанных в разделе 1.1. Центральное понятие системы называется рабочим пространством. Рабочее пространство соответствует одной компании: в нём хранятся её клиенты, заказы, оплаты, прайс-лист и журнал действий. Пользователь может состоять в нескольких рабочих пространствах, например работать в одной студии и параллельно вести собственные заказы. Данные разных пространств полностью разделены, и сотрудник одной компании не должен никаким способом увидеть данные другой.

Функциональные требования к системе:

– регистрация с подтверждением электронной почты, вход, выход и восстановление пароля;

– ведение базы клиентов с поиском, архивом и проверкой на дубликаты по телефону и почте;

– создание заказов с суммой, валютой, сроком, ответственным сотрудником и заметками;

– смена статуса заказа только по допустимым переходам;

– учёт частичных и полных оплат с автоматическим расчётом остатка и возможностью отменить ошибочный платёж;

– приглашение сотрудников и назначение им ролей с разными правами;

– напоминания о приближающихся и просроченных сроках, а также о неоплаченных выполненных заказах;

– сводная панель и отчёты за период с выгрузкой в CSV;

– журнал действий, в котором видно, кто и что изменил.

Кроме функций, к системе есть требования, которые касаются качества. Самое строгое из них относится к деньгам: суммы должны считаться точно, до тиына, без ошибок округления, и сервер не должен доверять суммам, посчитанным в браузере. Второе требование связано с доступом. Права пользователя проверяются на сервере при каждом запросе, а интерфейс только скрывает недоступные кнопки. Третье требование касается удобства: приложение должно нормально работать на экране телефона, потому что владелец мастерской часто проверяет заказы не за компьютером. Наконец, для малого бизнеса важна цена, поэтому систему должно быть можно разместить на бесплатных тарифах облачных сервисов.

Отдельно стоит сказать о ролях. После анализа типичной небольшой команды я выделяю три роли. Владелец (OWNER) управляет составом команды и видит всё. Менеджер (MANAGER) ведёт клиентов, заказы и оплаты, но не может менять состав команды. Сотрудник (EMPLOYEE) видит только назначенные ему заказы и может продвигать их по статусам, но не может менять суммы или отмечать оплаты. Такое разделение покрывает большинство ситуаций и при этом не перегружает настройки.

1.4 Выбор технологий

Приложение построено по схеме, в которой клиентская часть и серверная часть разделены. Клиентская часть представляет собой одностраничное приложение (SPA), которое загружается в браузер один раз и дальше обменивается с сервером только данными в формате JSON. Серверная часть предоставляет REST API, то есть набор адресов, где каждый адрес соответствует ресурсу (клиенту, заказу, оплате), а действие определяется HTTP-методом: GET для чтения, POST для создания, PUT и PATCH для изменения [6]. Такое разделение позволяет менять интерфейс, не трогая бизнес-логику, а все проверки держать в одном месте, на сервере.

Для серверной части выбран язык Java 21 и фреймворк Spring Boot [7]. Главный аргумент для меня здесь связан с деньгами. В Java есть класс BigDecimal, который хранит десятичные числа точно, тогда как обычные типы с плавающей точкой (double в Java, Number в JavaScript) не могут точно представить даже число 0,1. Дж. Блох прямо рекомендует не использовать float и double там, где нужен точный ответ, особенно в денежных расчётах [8]. Вторая причина в том, что Spring Security уже содержит проверенные механизмы для сессий, защиты от CSRF и хеширования паролей [9], и их не нужно писать самому. Кроме того, строгая типизация Java находит многие ошибки ещё при компиляции.

В качестве базы данных используется PostgreSQL 17 [10]. Данные ClientFlow по своей природе реляционные: заказ принадлежит клиенту,

оплата принадлежит заказу, сотрудник состоит в рабочем пространстве. Реляционная база умеет проверять эти связи сама через внешние ключи, а ограничения CHECK не дают записать, например, отрицательную сумму. Для учёта оплат особенно важны транзакции и блокировки строк, о которых подробнее пойдёт речь в главе II. Документные базы вроде MongoDB или Firebase дают больше свободы в структуре данных, но связи и денежные правила в них пришлось бы проверять в коде приложения. Для размещения базы выбран сервис Supabase, у которого есть бесплатный тариф с PostgreSQL [11].

Структура базы меняется через миграции Flyway [12]. Каждое изменение записано в отдельный SQL-файл с номером версии (V1, V2 и так далее), и при запуске приложение само применяет недостающие миграции. Hibernate при этом настроен только на проверку того, что классы Java совпадают со схемой базы, а не на её автоматическое изменение. Так я всегда точно знаю, какой SQL выполнится на рабочей базе.

Клиентская часть написана на React 19 и TypeScript [13]. Сборкой занимается Vite, стили описаны с помощью Tailwind CSS, а готовые элементы интерфейса (кнопки, диалоги, таблицы) взяты из набора shadcn/ui. Для загрузки данных с сервера используется TanStack Query [14]: библиотека кэширует ответы, показывает состояние загрузки и после изменения данных сама обновляет нужные списки. Формы проверяются библиотеками react-hook-form и zod, а переводы интерфейса подключены через i18next.

Отдельного решения потребовал способ входа в систему. Во многих учебных примерах после входа сервер выдаёт JWT-токен, который хранится в localStorage браузера. Мне этот вариант не подходит по двум причинам. Во-первых, если на страницу попадёт вредоносный скрипт (XSS-атака), он сможет прочитать localStorage и унести токен. Во-вторых, выданный JWT нельзя отозвать до истечения срока его действия, а пользователю нужна возможность выйти на всех устройствах сразу или завершить чужую сессию. Поэтому в ClientFlow используются серверные сессии: браузер получает cookie с флагом HttpOnly, который недоступен JavaScript, а сами сессии хранятся в PostgreSQL через Spring Session. За это приходится платить необходимостью защиты от подделки межсайтовых запросов (CSRF), которая описана в разделе 3.1 [15].

Итоговый набор технологий приведён в таблице 2.

Таблица 2. Технологии, использованные в проекте

Часть системы	Технологии	Назначение
Сервер	Java 21, Spring Boot 4.1, Spring Security 7, Spring Data JPA, Spring Session JDBC	REST API, бизнес-правила, проверка прав, сессии
База данных	PostgreSQL 17, Flyway	хранение данных, ограничения целостности, миграции схемы
Интерфейс	React 19, TypeScript, Vite, Tailwind CSS, shadcn/ui	одностраничное веб-приложение
Работа с	TanStack Query, React Router, react-	загрузка и кэширование,

Часть системы	Технологии	Назначение
данными в интерфейсе	hook-form, zod, i18next	маршруты, формы, переводы
Тестирование	JUnit 5, Spring Boot Test, Testcontainers, Vitest, React Testing Library	автоматические тесты сервера и интерфейса
Развёртывание	Docker, nginx, Vercel, Supabase, Gmail SMTP	локальный запуск и размещение в интернете

ГЛАВА II. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ CLIENTFLOW

2.1 Архитектура приложения

ClientFlow состоит из трёх частей: интерфейса в браузере, сервера с REST API и базы данных. Общая схема показана на рисунке 1.

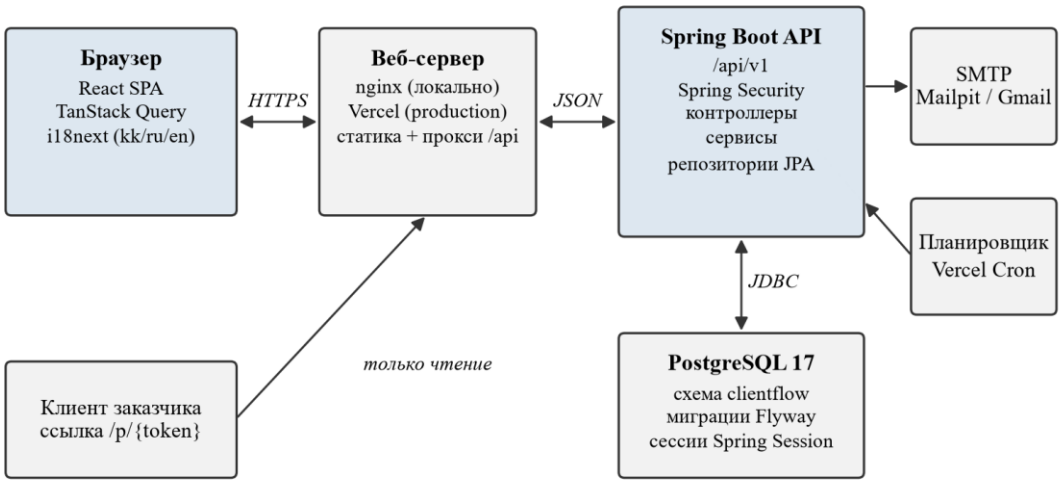


Рисунок 1. Архитектура приложения ClientFlow

Браузер загружает интерфейс с веб-сервера. При локальном запуске эту роль выполняет nginx, а в рабочей версии сервис Vercel. Все запросы, адрес которых начинается с /api, веб-сервер передаёт дальше на сервер приложения. Для браузера интерфейс и API находятся на одном домене, поэтому cookie сессии отправляется автоматически и не нужно открывать междоменные запросы (CORS). Сервер приложения обращается к PostgreSQL, отправляет письма через SMTP и раз в сутки получает вызов от планировщика, который запускает рассылку напоминаний и очистку устаревших записей. Отдельно на схеме показан заказчик, который открывает публичную ссылку на свой заказ. Ему доступно только чтение, и вход в систему для этого не нужен.

Внутри сервер разделён на три слоя, что соответствует классической схеме корпоративных приложений [16]. Контроллеры принимают HTTP-запрос, проверяют формат входных данных и возвращают ответ. Сервисы содержат бизнес-правила и управляют транзакциями: именно в сервисе проверяется, можно ли перевести заказ в новый статус или принять оплату. Репозитории отвечают за чтение и запись в базу данных. Зависимости направлены в одну сторону, от контроллера к репозиторию, поэтому правила работы с деньгами не зависят от того, каким способом пришёл запрос [17].

Код разложен по пакетам не по слоям, а по функциям: auth (вход и регистрация), client, order, payment, catalog (прайс-лист), report, notification, workspace, share (ссылки для клиентов), user, activity (журнал действий), а

также общие пакеты common, config и mail. Мне так удобнее: всё, что относится к оплатам, лежит в одном пакете payment, и при изменении правил не нужно искать связанные классы по разным папкам. Всего серверная часть занимает около 7,8 тысячи строк на Java, а клиентская около 8,4 тысячи строк на TypeScript без учёта стандартных компонентов shadcn/ui.

Ошибки сервер возвращает в едином формате JSON с кодом, например OVERPAYMENT или STALE_VERSION, и, если ошибка касается формы, со списком полей. Текст ошибки сервер не формирует. Интерфейс получает код и сам показывает сообщение на выбранном пользователем языке. Трассировки стека и внутренние сообщения исключений наружу не передаются, это отключено в настройках Spring Boot.

2.2 Модель данных

База данных содержит 17 таблиц приложения и две служебные таблицы Spring Session, в которых хранятся сессии пользователей. Основные таблицы и связи между ними показаны на рисунке 2. Служебные таблицы на схему не вынесены.

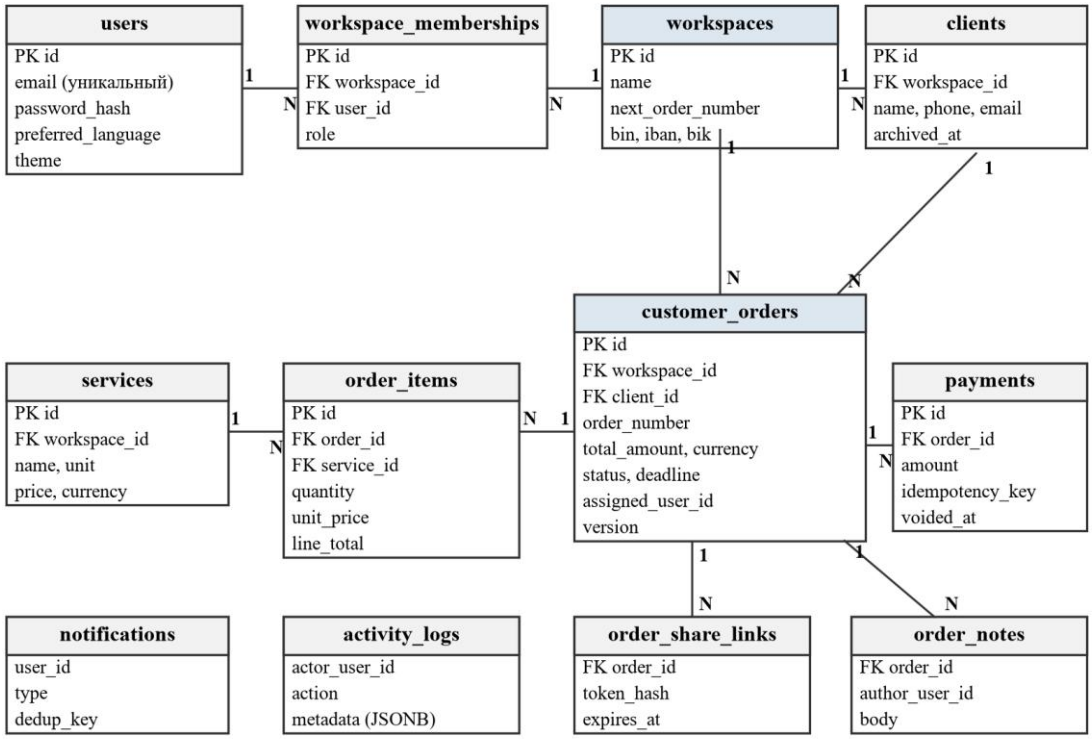


Рисунок 2. Основные таблицы базы данных и связи между ними

Главное правило при проектировании схемы звучит так: база данных должна сама не допускать некорректных состояний, а не полагаться только на код приложения [18]. Несколько решений из этой схемы стоит объяснить подробнее.

Первичные ключи всех основных таблиц имеют тип UUID, а не последовательное число. Если бы заказ открывался по адресу /orders/15, любой пользователь мог бы попробовать адреса /orders/14 и /orders/16 и узнать, сколько заказов есть в системе. UUID угадать невозможно. Для

людей у заказа есть отдельный понятный номер вида ORD-000001, который считается внутри каждого рабочего пространства.

Денежные суммы хранятся в типе NUMERIC(14,2), то есть с точностью до двух знаков после запятой и до 999 миллиардов в целой части. Моменты времени хранятся в типе TIMESTAMPTZ в часовом поясе UTC. Сроки заказов и периоды отчётов при этом считаются в часовом поясе Asia/Almaty, иначе заказ со сроком «сегодня» стал бы просроченным в пять часов утра по местному времени.

Самым интересным решением мне кажется составной внешний ключ. Таблица заказов ссылается на клиента не просто по его идентификатору, а по паре (client_id, workspace_id). Это значит, что база физически не даст создать заказ, клиент которого принадлежит другому рабочему пространству, даже если в коде сервиса будет ошибка. Так же связаны оплаты, позиции, заметки и ссылки с заказами. Изоляция данных между компаниями обеспечивается одновременно в коде и на уровне схемы.

Клиентов и заказы в ClientFlow не удаляют, а переносят в архив, для этого в таблицах есть поле archived_at. Ограничение CHECK разрешает архивировать только выполненные или отменённые заказы. Платежи нельзя ни удалить, ни изменить: их можно только отменить с указанием причины. Другие ограничения CHECK проверяют, что email записан строчными буквами, валюта входит в список KZT, RUB, USD, EUR, БИН состоит ровно из 12 цифр, а IBAN соответствует своему формату.

Схема создавалась постепенно, по мере добавления функций. Каждое изменение оформлено отдельной миграцией Flyway (таблица 3).

Таблица 3. Миграции базы данных

Миграция	Содержание
V1	основная схема: пользователи, рабочие пространства, участники, токены, клиенты, заказы, заметки, оплаты, уведомления, журнал действий
V2	таблицы Spring Session для хранения сессий в PostgreSQL
V3	отзыв прав у ролей anon и authenticated, через которые Supabase открывает данные по своему API
V4	таблица счётчиков для ограничения частоты запросов
V5	приглашения сотрудников в рабочее пространство
V6	аватары пользователей и каскадное удаление при удалении пространства
V7	прайс-лист услуг и позиции заказа
V8	реквизиты компании для счёта на оплату (БИН, ИИК/IBAN, БИК)
V9	публичные ссылки на заказ для клиентов

2.3 Рабочие пространства и роли

При регистрации пользователь сразу получает собственное рабочее пространство, в котором он является владельцем. Пригласить сотрудника можно по адресу электронной почты. Если человек с таким адресом уже зарегистрирован и подтвердил почту, он сразу становится участником. Если нет, приглашение сохраняется и срабатывает, когда этот человек зарегистрируется и подтвердит адрес. В обоих случаях сервер отвечает

одинаково, чтобы через форму приглашения нельзя было проверить, зарегистрирован ли в системе тот или иной email.

Права трёх ролей, о которых шла речь в разделе 1.3, сведены в таблицу 4.

Таблица 4. Права ролей в рабочем пространстве

Действие	OWNER	MANAGER	EMPLOYEE
Управление командой, переименование пространства	да	нет	нет
Создание, изменение и архивирование клиентов	да	да	только просмотр
Создание, изменение и архивирование заказов	да	да	нет
Какие заказы видит	все	все	только назначенные ему
Смена статуса заказа	любой допустимый переход	любой допустимый переход	только вперёд и только у своих заказов
Учёт и отмена оплат, отчёты, журнал действий	да	да	нет

Адрес почти каждого запроса содержит идентификатор рабочего пространства, например /api/v1/workspaces/{ws}/orders. На каждом запросе сервер проверяет, что текущий пользователь является участником этого пространства, и читает его роль из базы данных. Роль специально не хранится в сессии. Если владелец понизил менеджера до сотрудника, новые права действуют уже на следующем запросе, без повторного входа.

Ещё одна деталь касается ответа на запрос к чужому пространству. Сервер возвращает код 404 «не найдено», а не 403 «доступ запрещён». Ответ 403 подтвердил бы, что пространство с таким идентификатором существует. Сотрудник, удалённый из команды, сразу теряет доступ, а назначенные ему заказы остаются без исполнителя, чтобы менеджер мог передать их другому человеку. Владелец не может удалить сам себя или снять с себя роль владельца, иначе пространство осталось бы без управления.

2.4 Заказы и их жизненный цикл

Заказ всегда привязан к клиенту из того же рабочего пространства. При создании указываются название, описание, сумма, валюта, срок и ответственный сотрудник. Номер заказа выдаётся автоматически. Для этого в таблице рабочих пространств есть счётчик next_order_number. При создании заказа строка пространства блокируется, счётчик увеличивается на единицу, и блокировка снимается вместе с завершением транзакции. Стандартная последовательность PostgreSQL здесь не подходит, потому что она общая для всей таблицы, а номера должны идти по порядку отдельно в каждой компании. Тест, который создаёт заказы параллельно, подтверждает, что номера не повторяются и не пропускаются.

Статус заказа меняется по фиксированной схеме, показанной на рисунке 3.

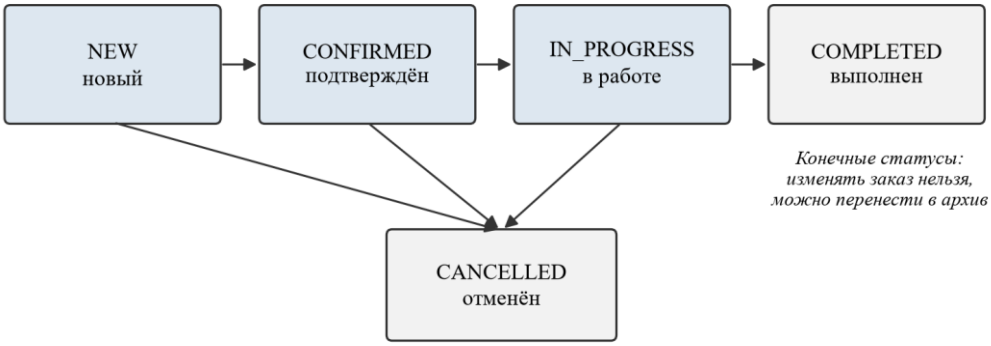


Рисунок 3. Допустимые переходы между статусами заказа

Новый заказ можно подтвердить, подтверждённый взять в работу, а заказ в работе отметить выполненным. Отменить заказ можно на любом из трёх первых этапов. Переход из статуса «в работе» в «отменён» в первоначальном описании задачи отсутствовал. Он был добавлен осознанно, потому что начатую работу на практике тоже иногда отменяют.

Выполненные и отменённые заказы считаются конечными, изменить их уже нельзя, можно только перенести в архив. Правила переходов описаны в одном месте, в перечислении OrderStatus, и проверяются на сервере. Сотрудник может перемещать свои заказы только вперёд по цепочке «подтверждён, в работе, выполнен» и не может их отменить.

Отдельная проблема возникает, когда один заказ одновременно редактируют два менеджера. Допустим, первый открыл заказ и меняет срок, а второй в это время открыл ту же карточку и меняет сумму. Без специальной защиты тот, кто сохранит позже, молча затрёт изменения первого. Для этого в таблице заказов есть поле version, а метод называется оптимистической блокировкой [16]. Интерфейс отправляет вместе с изменениями номер версии, который он видел. Если за это время заказ уже изменили, версия в базе больше, и сервер отвечает ошибкой 409 с кодом STALE_VERSION. Пользователь видит сообщение, что данные устарели, и может обновить карточку.

Заказ считается просроченным, если его срок меньше сегодняшней даты по времени Алматы, а статус ещё не конечный. К каждому заказу можно оставлять заметки, а все действия с ним (создание, изменение полей, смена статуса, оплаты) попадают в журнал. Записи журнала хранят дополнительные сведения в поле типа JSONB, например старое и новое значение суммы, поэтому на странице заказа видна полная история.

Заказы можно просматривать таблицей с фильтрами по статусу, состоянию оплаты, клиенту и сроку или в виде доски, где они разложены по колонкам статусов. Во время тестирования поиска нашлась небольшая, но неприятная ошибка: символы % и _ в строке поиска база воспринимала как

шаблоны, и запрос «50%» находил лишнее. Теперь пользовательский текст экранируется и ищется буквально.

2.5 Учёт оплат

Учёт оплат я считаю самой ответственной частью системы, потому что ошибка здесь означает потерянные или лишние деньги. Правила, которые реализованы на сервере, выглядят так:

– оплачено = сумма всех неотменённых платежей по заказу;

– остаток = сумма заказа минус оплачено;

– состояние оплаты: UNPAID, если не оплачено ничего, PARTIALLY_PAID, если оплачена часть, и PAID, если оплачено полностью;

– сумма платежа больше нуля и не превышает остаток, а дата платежа не может быть в будущем;

– платёж можно принять только по заказу, который не отменён и не находится в архиве.

Остаток долга нигде не хранится, он каждый раз вычисляется. Если бы остаток хранился в отдельном поле, его пришлось бы пересчитывать при каждой оплате, отмене платежа или изменении суммы заказа, и рано или поздно где-то это забыли бы сделать. Сумма неотменённых платежей всегда даёт правильный результат.

На сервере все суммы имеют тип BigDecimal. Интерфейс передаёт суммы строками, например "12500.50", а не числами, чтобы JavaScript не превратил их в число с плавающей точкой. Там, где форма показывает предварительный расчёт (например, итог позиций заказа), он выполняется в целых сотых долях через тип bigint. Окончательную сумму всё равно пересчитывает сервер.

Самая сложная ситуация связана с одновременными платежами. Пусть заказ стоит 100 000 тенге, из них уже оплачено 60 000, и остаток равен 40 000. Два менеджера почти одновременно вносят по 30 000. Если ничего не предпринимать, оба запроса прочитают оплаченные 60 000, оба решат, что 30 000 меньше остатка, и оба сохранят платёж. В итоге клиент окажется переплатившим 20 000. Эта ошибка относится к классу состояний гонки, и проверка «прочитать, сравнить, записать» её не предотвращает [19].

В ClientFlow эта проблема решается блокировкой строки заказа командой SELECT ... FOR UPDATE [10]. Первая транзакция блокирует заказ, проверяет остаток и сохраняет платёж. Вторая транзакция в это время ждёт. Когда первая завершится, вторая прочитает уже 90 000 оплаченных, увидит, что 30 000 больше остатка в 10 000, и получит ошибку OVERPAYMENT. Порядок действий при записи платежа показан на рисунке 4, а фрагмент кода приведён в приложении А.

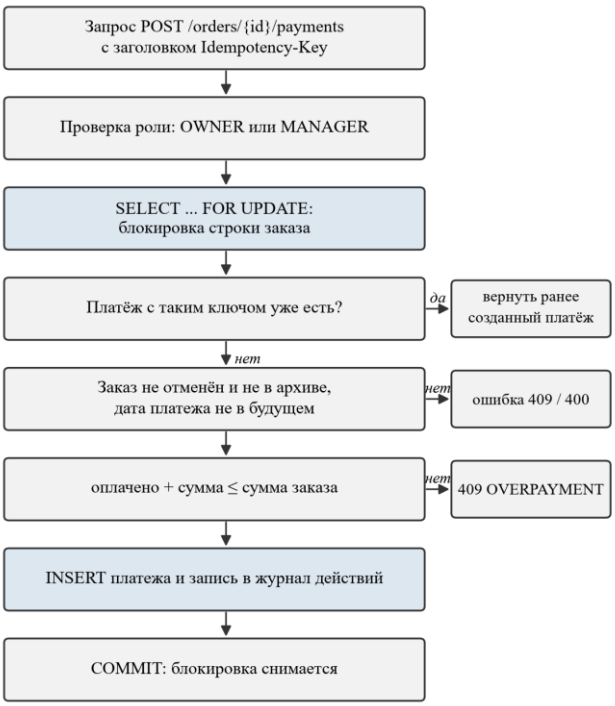


Рисунок 4. Порядок записи платежа на сервере

Есть и более простая, но частая ситуация: пользователь дважды нажимает кнопку «Сохранить» или запрос повторяется из-за плохого интернета. Для защиты от этого интерфейс генерирует для каждой попытки оплаты уникальный ключ (UUID) и передаёт его в заголовке Idempotency-Key. При повторной отправке той же попытки ключ не меняется. В базе на пару (workspace_id, idempotency_key) стоит уникальный индекс, а сервер, получив уже известный ключ, возвращает созданный ранее платёж вместо нового. Понятие идемпотентности, когда повторный запрос даёт тот же результат, что и первый, описано в спецификации HTTP [20]. Если тот же ключ пришёл для другого заказа, сервер отвечает ошибкой, чтобы не связать платёж не с тем заказом.

Исправить ошибочный платёж можно только отменой. Отменённый платёж остаётся в истории, но больше не учитывается в сумме оплаченного. При отмене обязательно указывается причина, а система сохраняет, кто и когда её выполнил. Ограничение в базе следит, чтобы дата отмены и причина были заполнены одновременно. С оплатами связаны ещё три правила для заказа. Сумму заказа нельзя сделать меньше уже оплаченной. Валюту нельзя изменить, если по заказу есть платежи. Заказ с действующими платежами нельзя отменить, пока эти платежи не отменены, что соответствует возврату денег клиенту.

2.6 Дополнительные возможности

Когда основной учёт заработал, в приложение были добавлены функции, которые приближают его к реальной работе небольшой компании.

Прайс-лист и позиции заказа. В каждом рабочем пространстве можно вести список услуг с единицей измерения и ценой. Заказ можно составить из позиций: выбрать услугу из прайс-листа или ввести произвольную строку. Сумма строки считается как количество, умноженное на цену, с округлением до сотых по правилу HALF_UP, а сумма заказа равна сумме строк. Считает её только сервер. Если позиция взята из прайс-листа, сервер берёт цену из каталога и игнорирует цену, присланную браузером. Позиция хранит копию названия и цены на момент сохранения, поэтому последующее изменение прайс-листа не меняет уже оформленные заказы.

Счёт на оплату. Владелец заполняет реквизиты компании: юридическое название, БИН, адрес, банк, ИИК (IBAN) и БИК. По любому заказу можно открыть печатную форму счёта и сохранить её в PDF через стандартную печать браузера. Генерировать PDF на сервере не понадобилось: печать браузера даёт тот же результат без дополнительных библиотек.

Ссылка для клиента. По заказу можно создать публичную ссылку вида /p/{token}, по которой заказчик без регистрации видит статус, позиции и свои оплаты. Внутренние заметки, сотрудники и контакты по этой ссылке не передаются. Токен имеет длину 256 бит, в базе хранится только его хеш SHA-256, срок действия ссылки 90 дней, и её можно отозвать в любой момент. Количество обращений к публичным ссылкам с одного IP-адреса ограничено 60 запросами в минуту.

Импорт клиентов из CSV. Многие начинают работу с уже существующей таблицей клиентов, поэтому её можно загрузить файлом. Сначала сервер показывает предварительный просмотр: какие строки будут добавлены, какие содержат ошибки и какие повторяют уже существующих клиентов или друг друга. После подтверждения все строки добавляются в одной транзакции. Поддерживаются кодировки UTF-8 и Windows-1251 (в ней сохраняет CSV русскоязычный Excel), разделители «;» и «,» и заголовки столбцов на трёх языках. Размер файла ограничен 1 МБ, а количество строк 1000.

Уведомления. Система создаёт напоминания четырёх типов: о сроке, до которого осталось не больше двух дней, о просроченном заказе, об изменении заказа и о выполненном, но не полностью оплаченном заказе (раз в неделю). Напоминания о сроке получает ответственный сотрудник, а если его нет, то владелец и менеджеры. Напоминания создаются несколькими SQL-запросами вида INSERT ... SELECT, которые обрабатывают все рабочие пространства сразу, без загрузки заказов в память. У каждого напоминания есть ключ, например DEADLINE:<номер заказа>:<срок>, и уникальный индекс по паре (пользователь, ключ). Поэтому повторный запуск задачи, даже на двух серверах одновременно, не создаёт дублей. Любой тип уведомлений пользователь может отключить в настройках.

Сводная панель, отчёты и календарь. На главной странице показаны количество клиентов, активных, выполненных и просроченных заказов, распределение заказов по статусам, полученные оплаты и общая сумма долга за выбранный период, ближайшие сроки и последние действия в команде.

Суммы группируются по валютам, потому что складывать тенге с долларами нельзя. Для сотрудника все цифры считаются только по его заказам. Отчёты по заказам и оплатам можно выгрузить в CSV. В начало файла добавляется метка кодировки, чтобы Excel правильно открыл казахский и русский текст, а ячейки, начинающиеся с символов =, +, - или @, экранируются. Без этого Excel мог бы выполнить такую ячейку как формулу, что известно как CSV-инъекция [21]. Кроме того, есть график поступлений по месяцам за последний год и календарь сроков заказов.

2.7 Пользовательский интерфейс

Интерфейс ClientFlow представляет собой одностраничное приложение с боковым меню. Основные страницы перечислены в таблице 5.

Таблица 5. Основные страницы приложения

Адрес	Назначение
/login, /register, /forgot-password	вход, регистрация и восстановление пароля
/dashboard	сводная панель с показателями за период
/orders, /orders/:id	список и доска заказов, карточка заказа с позициями, оплатами, заметками и историей
/orders/:id/invoice	печатная форма счёта на оплату
/clients, /clients/:id	база клиентов, карточка клиента с его заказами
/services	прайс-лист услуг
/calendar	календарь сроков заказов
/reports	отчёты по заказам и оплатам с выгрузкой в CSV
/team	участники команды и приглашения
/settings	профиль, пароль, активные сессии, язык, тема, уведомления, реквизиты
/p/:token	публичная страница заказа для клиента

Интерфейс доступен на казахском, русском и английском языках. Все тексты вынесены в три файла переводов, и отдельный тест проверяет, что в каждом из них есть одинаковый набор ключей, а для каждого языка заданы все нужные формы множественного числа. Это оказалось полезным: в казахском и английском две формы («1 заказ», «2 заказа»), а в русском их больше («1 заказ», «2 заказа», «5 заказов»), и без проверки такие формы легко забыть. Выбранный язык сохраняется в профиле, и на этом же языке приходят письма.

Приложение поддерживает светлую и тёмную тему, а также режим, который повторяет настройку операционной системы. Тема применяется маленьким скриптом ещё до отрисовки страницы, поэтому при загрузке экран не мигает белым. На телефоне боковое меню превращается в выдвижную панель, таблицы прокручиваются по горизонтали, а календарь показывает сроки списком вместо сетки месяца.

Формы проверяются ещё в браузере, и сообщение об ошибке появляется прямо под полем. Если ошибку нашёл сервер (например, такой email уже занят), интерфейс получает код поля и показывает сообщение в

том же месте. Опасные действия, такие как отмена заказа, отмена платежа или удаление аккаунта, требуют подтверждения в отдельном диалоге. Логотип приложения (рисунок 5) составлен из буквы С, силуэтов клиентов и листа заказа с отметкой о выполнении, то есть из тех трёх вещей, вокруг которых построена вся система.



Рисунок 5. Логотип приложения ClientFlow

ГЛАВА III. БЕЗОПАСНОСТЬ, ТЕСТИРОВАНИЕ И ВНЕДРЕНИЕ СИСТЕМЫ

3.1 Защита учётных записей и данных

Для проверки безопасности я ориентируюсь на список OWASP Top 10, в котором собраны самые распространённые уязвимости веб-приложений [22]. Первое место в этом списке занимает нарушение контроля доступа, то есть ситуация, когда пользователь получает данные, которые ему видеть не положено. Как в ClientFlow устроена проверка прав и изоляция рабочих пространств, описано в разделах 2.2 и 2.3. Ниже речь пойдёт об остальных мерах защиты.

Пароли хранятся не в открытом виде, а в виде хеша, вычисленного алгоритмом Argon2id. Это современный алгоритм, который специально требует много памяти, поэтому подбирать пароли на видеокартах оказывается дорого [23]. OWASP рекомендует его как первый вариант для хранения паролей [24]. При регистрации пользователь должен подтвердить адрес электронной почты, и пока он этого не сделал, войти в систему нельзя.

Ссылки из писем для подтверждения почты и сброса пароля содержат случайный токен. В базе хранится не сам токен, а только его хеш SHA-256, поэтому даже при утечке базы ссылками воспользоваться не получится. Токен подтверждения действует 24 часа, токен сброса пароля 30 минут, и каждый из них можно использовать только один раз. После смены пароля через сброс все активные сессии пользователя завершаются.

Система не раскрывает, какие адреса в ней зарегистрированы. При неверном пароле и при несуществующем email пользователь получает одно и то же сообщение. Повторная регистрация на уже занятый адрес и запрос сброса пароля для неизвестного адреса выглядят для посетителя так же, как успешные. Иначе злоумышленник мог бы проверить список адресов и узнать, кто пользуется сервисом.

Чтобы пароль нельзя было подобрать перебором, количество попыток ограничено. Счётчики хранятся в PostgreSQL, а не в памяти сервера. В бесплатном облачном размещении может одновременно работать несколько экземпляров сервера, и счётчик в памяти каждого из них был бы своим. Ограничение реализовано одним атомарным запросом INSERT ... ON CONFLICT DO UPDATE, который увеличивает счётчик и сбрасывает его, когда истекло окно времени. Действующие лимиты приведены в таблице 6.

Таблица 6. Ограничения частоты запросов

Действие	Ограничение
Вход в систему	10 попыток на один email и 30 попыток с одного IP-адреса за 15 минут
Регистрация	10 попыток с одного IP-адреса в час
Повторная отправка письма подтверждения	3 письма на один email и 10 запросов с одного IP-адреса в час
Запрос сброса пароля	3 письма на один email и 10 запросов с одного IP-адреса в час
Проверка пароля при опасных	10 попыток в час для одного пользователя

Действие	Ограничение
действиях	
Открытие публичной ссылки на заказ	60 запросов в минуту с одного IP-адреса

Cookie сессии имеет флаги HttpOnly и SameSite, а в рабочей версии ещё и Secure, то есть передаётся только по HTTPS. Сессия действует 7 дней. В настройках пользователь видит список своих активных сессий с браузером, IP-адресом и временем последней активности и может завершить любую из них или все, кроме текущей. После входа идентификатор сессии меняется, что защищает от атаки фиксации сессии.

Раз вход основан на cookie, нужна защита от подделки межсайтовых запросов (CSRF). Без неё чужой сайт мог бы заставить браузер пользователя отправить запрос к ClientFlow, и браузер сам приложил бы cookie. Spring Security выдаёт специальный токен в cookie XSRF-TOKEN, а интерфейс при каждом изменяющем запросе копирует его в заголовок X-XSRF-TOKEN [15]. Чужой сайт прочитать эту cookie не может и, следовательно, не может подделать заголовок. Если токен устарел, интерфейс один раз запрашивает новый и повторяет запрос, так что пользователь этого даже не замечает.

Веб-сервер отправляет заголовки безопасности: Content-Security-Policy, которая разрешает загружать скрипты только с собственного домена, а также запрет встраивать сайт во фрейм, Strict-Transport-Security и другие. Из-за строгой политики CSP скрипт выбора темы пришлось вынести в отдельный файл, так как встроенные скрипты политика запрещает.

Отдельная мера относится к Supabase. Этот сервис автоматически открывает таблицы схемы public через собственный API, а публичный ключ к этому API по определению не секретный. Поэтому все таблицы ClientFlow находятся в отдельной схеме clientflow, а миграция V3 отзывает у ролей anon и authenticated любой доступ к ней. Интерфейс вообще не использует ключи Supabase и получает данные только через сервер приложения. Пароли, ключи и адреса баз данных передаются через переменные окружения и в репозиторий не попадают.

3.2 Тестирование

Основная часть проверок автоматизирована. На стороне сервера используются JUnit 5 и Spring Boot Test. Тесты отправляют настоящие HTTP-запросы к приложению, как это делал бы браузер, включая cookie сессии и CSRF-токен, и проверяют ответы и состояние базы данных.

Для тестов поднимается настоящий PostgreSQL 17 в Docker-контейнере с помощью библиотеки Testcontainers [25]. Часто в учебных проектах для тестов берут встроенную базу H2, но в моём случае это не подходит. Блокировки SELECT ... FOR UPDATE, поле JSONB, ограничения CHECK с регулярными выражениями и INSERT ... ON CONFLICT в H2 работают иначе или не работают вовсе, и тест на H2 мог бы пройти там, где на PostgreSQL была бы ошибка. Рабочая база в тестах никогда не используется.

Серверные тесты разделены на классы по функциям (таблица 7). В них 121 тестовый метод. Один из методов параметризованный: он проверяет восемь недопустимых переходов статуса, поэтому при запуске выполняется 128 проверок.

Таблица 7. Автоматические тесты серверной части

Класс тестов	Что проверяется	Тестов
AuthFlowTests	регистрация, подтверждение почты, вход, выход, сброс пароля, ограничение попыток	17
PaymentTests	расчёт остатка, переплата, повторная отправка, параллельные платежи, отмена платежей	15
OrderRulesTests	переходы статусов, версии, номера заказов, просрочка, поиск	14
RolePermissionTests	права владельца, менеджера и сотрудника	10
ExtrasTests	счёт и реквизиты, публичные ссылки, импорт CSV, график выручки, календарь	10
ClientTests	клиенты, поиск, архив, поиск дубликатов	8
OrderItemTests	расчёт суммы по позициям, цена из каталога	8
NotificationTests	напоминания и отсутствие дублей	7
DashboardAndReportTests	сводная панель, отчёты, выгрузка CSV	7
InfrastructureTests	миграции, ответ 401 без перенаправления, защита от CSRF, заголовки безопасности, CORS	7
WorkspaceIsolationTests	недоступность данных чужого рабочего пространства	6
MembershipTests	приглашения и участники команды	6
AccountTests	профиль, смена пароля, сессии, удаление аккаунта	6

Самыми полезными оказались тесты на одновременные действия. Например, тест concurrentPaymentsCannotExceedTheOrderTotal одновременно, в шести параллельных потоках, отправляет шесть платежей, каждый из которых равен 60% стоимости заказа. Пройти может только один из них, и тест проверяет, что сервер принял ровно один платёж, а остальные пять отклонил с ошибкой 409. Тест parallelDuplicateSubmissionsCreateOnePayment одновременно отправляет один и тот же запрос с одинаковым ключом идемпотентности и проверяет, что в базе появился ровно один платёж. Такие ошибки почти невозможно поймать ручной проверкой, потому что они проявляются только при точном совпадении времени запросов.

Тесты изоляции написаны с точки зрения злоумышленника. Пользователь из одного пространства пытается открыть чужое пространство, прочитать или изменить чужого клиента по его идентификатору, подставить чужой идентификатор внутри своего пространства, создать заказ для чужого клиента или назначить исполнителем чужого сотрудника. Во всех случаях сервер должен ответить «не найдено», и тесты это проверяют.

Клиентская часть проверяется с помощью Vitest и React Testing Library. Тесты отрисовывают страницы в виртуальном браузере и проверяют

поведение с точки зрения пользователя: что форма входа показывает переведённую ошибку при неверном пароле, что при повторной отправке оплаты используется тот же ключ идемпотентности, что суммы округляются так же, как на сервере, что при недоступном сервере появляется сообщение с кнопкой повтора, что переводы полные. При последнем запуске 49 тестов в 9 файлах завершились успешно. Помимо этого, два сквозных сценария были проверены вручную в браузере Chrome поверх локального окружения docker-compose.

3.3 Развёртывание

Для локальной разработки весь проект запускается одной командой docker compose up. Она поднимает четыре контейнера: PostgreSQL, Mailpit, сервер приложения и nginx с собранным интерфейсом. Mailpit представляет собой почтовый ящик для разработки. Все письма с подтверждением и сбросом пароля попадают в него, и их можно открыть в браузере, при этом наружу ничего не отправляется.

Образ сервера собирается в два этапа. На первом этапе Maven компилирует проект и собирает jar-файл, на втором в итоговый образ попадают только среда выполнения Java и этот файл. Образ получается меньше, а приложение внутри контейнера запускается от отдельного пользователя без прав администратора. В образ встроена проверка работоспособности через адрес /actuator/health, который отражает состояние базы данных. Почтовый сервер в эту проверку специально не включён, чтобы временная недоступность SMTP не приводила к перезапуску всего приложения.

Для размещения в интернете выбрана схема из бесплатных сервисов. Интерфейс и контейнер сервера размещаются на Vercel [26], база данных на Supabase, а письма отправляются через SMTP-сервер Gmail с паролем приложения, что позволяет отправлять около 500 писем в сутки без собственного домена. Vercel Cron раз в сутки, в 03:00 по UTC (08:00 по времени Алматы), вызывает служебный адрес, защищённый секретным ключом, и запускает формирование напоминаний. На серверах, работающих постоянно, ту же задачу ежечасно выполняет встроенный планировщик Spring. Пул соединений с базой ограничен тремя соединениями, потому что каждый экземпляр сервера в облаке открывает собственные соединения, а их число на бесплатном тарифе Supabase ограничено. Адрес сервера в правиле проксирования Vercel нужно указать при первом развёртывании.

3.4 Проблемы при разработке и ограничения системы

Не все решения удалось принять с первого раза. Наиболее показательная проблема обнаружилась уже в конце работы. Если сервер был недоступен, приложение бесконечно показывало индикатор загрузки. Выяснилось, что после неудачной проверки сессии каждая вновь открытая страница заново запускала запрос, состояние сбрасывалось в «загрузка», и защита маршрутов по кругу переключалась между экраном загрузки и

страницей. Исправление состоит из двух частей: повторный запрос при открытии страницы отключён, а вместо бесконечной загрузки показывается сообщение «сервер недоступен» с кнопкой «Повторить». Для этого случая добавлен отдельный тест.

Похожая ситуация была с отправкой писем. Если SMTP-сервер долго не отвечал, регистрация зависала вместе с ним. Теперь у соединения с почтовым сервером установлены таймауты по 10 секунд. При ошибке она записывается в журнал, пользователь всё равно регистрируется и может запросить письмо повторно. По той же причине облачного размещения аватары до 512 КБ хранятся прямо в PostgreSQL: файловая система контейнера не сохраняется между перезапусками.

У текущей версии системы есть ограничения, о которых нужно сказать честно:

– закон о персональных данных требует, чтобы база с персональными данными граждан Казахстана находилась на территории республики [2]; серверы бесплатного тарифа Supabase расположены за рубежом, поэтому для реального коммерческого использования базу нужно перенести к казахстанскому провайдеру;

– оплаты фиксируются вручную, приёма платежей онлайн и интеграции с банками нет;

– уведомления показываются только внутри приложения, по SMS и в мессенджеры они не отправляются;

– нагрузочное тестирование не проводилось, поэтому поведение системы при сотнях одновременных пользователей не проверено;

– бесплатные тарифы облачных сервисов ограничивают число писем, соединений с базой и частоту фоновых задач.

Ни одно из этих ограничений не мешает использовать систему небольшой команде, но их нужно учитывать, если ClientFlow будет развиваться дальше.

3.5 Направления развития

Исходя из перечисленных ограничений, я вижу несколько направлений, в которых систему можно развивать. Самое полезное для пользователей, на мой взгляд, состоит в напоминаниях клиентам: письмо или сообщение в мессенджере о готовности заказа или о сумме к доплате, отправленное прямо из карточки заказа. Вторым шагом могла бы стать интеграция с популярными в Казахстане платёжными сервисами, чтобы клиент оплачивал заказ по ссылке, а платёж записывался автоматически.

С технической стороны стоит добавить двухфакторную аутентификацию для владельцев, перенести базу данных к казахстанскому провайдеру и провести нагрузочное тестирование. Интерфейс можно оформить как устанавливаемое веб-приложение (PWA), чтобы оно открывалось с главного экрана телефона.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была разработана система ClientFlow для учёта клиентов, заказов и оплат на малом предприятии. Цель работы достигнута: приложение позволяет нескольким сотрудникам совместно вести клиентов и заказы, учитывать частичные оплаты и видеть точный остаток долга по каждому заказу.

В первой главе рассмотрено, какие данные нужны небольшой компании для учёта заказов и с какими проблемами она сталкивается при ручном учёте: дубли клиентов, пропущенные сроки, ошибки в суммах, отсутствие истории изменений и общий доступ ко всем данным. Сравнение существующих способов учёта показало, что простые инструменты не защищают от ошибок, а профессиональные системы избыточны для маленькой команды. На основе этого сформулированы требования к системе и выбраны технологии: Java 21 и Spring Boot для сервера, PostgreSQL для базы данных, React и TypeScript для интерфейса.

Во второй главе описаны архитектура и модель данных, а также реализация основных функций. Данные разных компаний разделены с помощью рабочих пространств, причём изоляция обеспечивается и в коде, и составными внешними ключами в базе данных. Права трёх ролей проверяются на сервере при каждом запросе. Для заказов реализована схема статусов с фиксированными переходами и защита от одновременного редактирования через номер версии. Учёт оплат построен на точной десятичной арифметике, блокировке строки заказа при записи платежа и ключах идемпотентности, которые не дают создать один платёж дважды. Дополнительно реализованы прайс-лист, счёт на оплату, публичная ссылка для клиента, импорт клиентов из CSV, напоминания, отчёты и интерфейс на трёх языках.

В третьей главе описаны меры защиты: хеширование паролей алгоритмом Argon2id, одноразовые токены в письмах, ограничение частоты запросов, серверные сессии с защитой от CSRF и заголовки безопасности. Работа системы проверена 128 автоматическими проверками сервера на настоящем PostgreSQL и 49 тестами интерфейса. Для приложения подготовлены локальное окружение в Docker и схема размещения на бесплатных облачных сервисах.

Основной практический результат работы состоит в том, что правила учёта денег проверяет сама система, а не человек. Нельзя принять оплату больше остатка, нельзя незаметно удалить платёж, нельзя увидеть чужие данные. Для малого предприятия это означает меньше ошибок и споров с клиентами. Главным ограничением текущей версии я считаю размещение базы данных за пределами Казахстана, что для реального использования потребует переноса на местный хостинг. Также в дальнейшем в систему стоит добавить уведомления клиентам и приём оплат онлайн.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Предпринимательский кодекс Республики Казахстан от 29 октября 2015 года № 375-V ЗРК. [Электронный ресурс]. – URL: https://adilet.zan.kz/rus/docs/K1500000375

2. Закон Республики Казахстан от 21 мая 2013 года № 94-V «О персональных данных и их защите». [Электронный ресурс]. – URL: https://adilet.zan.kz/rus/docs/Z1300000094

3. Закон Республики Казахстан от 24 ноября 2015 года № 418-V «Об информатизации». [Электронный ресурс]. – URL: https://adilet.zan.kz/rus/docs/Z1500000418

4. Битрикс24: официальный сайт. [Электронный ресурс]. – URL: https://www.bitrix24.kz/

5. amoCRM: официальный сайт. [Электронный ресурс]. – URL: https://www.amocrm.ru/

6. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: doctoral dissertation. – Irvine: University of California, 2000. [Электронный ресурс]. – URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

7. Spring Boot Reference Documentation. [Электронный ресурс]. – URL: https://docs.spring.io/spring-boot/

8. Блох Дж. Java: эффективное программирование. 3-е изд. – М.: Диалектика, 2019. – 464 с.

9. Spring Security Reference. [Электронный ресурс]. – URL: https://docs.spring.io/spring-security/reference/

10. PostgreSQL 17 Documentation. [Электронный ресурс]. – URL: https://www.postgresql.org/docs/17/

11. Supabase Documentation. [Электронный ресурс]. – URL: https://supabase.com/docs

12. Flyway Documentation. [Электронный ресурс]. – URL: https://documentation.red-gate.com/flyway

13. React Documentation. [Электронный ресурс]. – URL: https://react.dev/

14. TanStack Query Documentation. [Электронный ресурс]. – URL: https://tanstack.com/query/latest

15. OWASP Cheat Sheet Series. Cross-Site Request Forgery Prevention Cheat Sheet. [Электронный ресурс]. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

16. Фаулер М. Архитектура корпоративных программных приложений. – М.: Вильямс, 2006. – 544 с.

17. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2018. – 352 с.

18. Дейт К. Дж. Введение в системы баз данных. 8-е изд. – М.: Вильямс, 2005. – 1328 с.

19. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб.: Питер, 2018. – 640 с.

20. RFC 9110. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – IETF, 2022. [Электронный ресурс]. – URL: https://www.rfc-editor.org/rfc/rfc9110

21. OWASP Foundation. CSV Injection. [Электронный ресурс]. – URL: https://owasp.org/www-community/attacks/CSV_Injection

22. OWASP Top 10:2021. [Электронный ресурс]. – URL: https://owasp.org/Top10/

23. RFC 9106. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications / A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. – IRTF, 2021. [Электронный ресурс]. – URL: https://www.rfc-editor.org/rfc/rfc9106

24. OWASP Cheat Sheet Series. Password Storage Cheat Sheet. [Электронный ресурс]. – URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

25. Testcontainers for Java. Documentation. [Электронный ресурс]. – URL: https://java.testcontainers.org/

26. Vercel Documentation. [Электронный ресурс]. – URL: https://vercel.com/docs

ПРИЛОЖЕНИЯ

Приложение А

Метод записи платежа (класс PaymentService)

```
@Transactional
public RecordResult record(WorkspaceAccess access, UUID orderId,
                           UUID idempotencyKey, RecordPaymentRequest request)
{
    requireManager(access);
    // SELECT ... FOR UPDATE: параллельные платежи по заказу идут по очереди
    CustomerOrder order = orders.findForUpdate(orderId, access.workspaceId())
            .orElseThrow(ApiException::notFound);

    // Повторная отправка (двойной клик, повтор запроса):
    // возвращается платёж, созданный в первый раз
    var existing = payments.findByWorkspaceIdAndIdempotencyKey(
            access.workspaceId(), idempotencyKey);
    if (existing.isPresent()) {
        if (!existing.get().getOrderId().equals(orderId)) {
            throw ApiException.conflict("IDEMPOTENCY_KEY_REUSED");
        }
        return new RecordResult(PaymentResponse.from(existing.get()), false);
    }

    if (!acceptsPayments(order)) {
        throw ApiException.conflict("ORDER_NOT_PAYABLE");
    }
    if (request.paymentDate().isAfter(LocalDate.now(clock))) {
        throw ApiException.badRequest("PAYMENT_DATE_IN_FUTURE");
    }
    BigDecimal paid = payments.sumValid(access.workspaceId(), orderId);
    if (paid.add(request.amount()).compareTo(order.getTotalAmount()) > 0) {
        throw ApiException.conflict("OVERPAYMENT");
    }

    Payment payment = payments.save(new Payment(access.workspaceId(),
            orderId, request.amount(), request.paymentMethod(),
            request.paymentDate(),
            blankToNull(request.reference()), idempotencyKey,
            users.getReferenceById(access.userId())));

    Map<String, Object> metadata = new LinkedHashMap<>();
    metadata.put("paymentId", payment.getId().toString());
    metadata.put("amount", payment.getAmount().toPlainString());
    metadata.put("currency", order.getCurrency());
    metadata.put("method", payment.getPaymentMethod().name());
    activity.record(access, "PAYMENT_RECORDED",
            ActivityLog.EntityType.ORDER, orderId, metadata);
    return new RecordResult(PaymentResponse.from(payment), true);
}
```

Фрагмент миграции V1: таблицы заказов и оплат

```
CREATE TABLE customer_orders (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id      UUID          NOT NULL REFERENCES workspaces (id)
                                    ON DELETE CASCADE,
    client_id         UUID          NOT NULL,
    order_number      VARCHAR(20)   NOT NULL,
    title             VARCHAR(200)  NOT NULL,
    description       VARCHAR(5000),
    total_amount      NUMERIC(14, 2) NOT NULL CHECK (total_amount > 0),
    currency          CHAR(3)       NOT NULL
                      CHECK (currency IN ('KZT', 'RUB', 'USD', 'EUR')),
    status            VARCHAR(12)   NOT NULL DEFAULT 'NEW'
                      CHECK (status IN ('NEW', 'CONFIRMED', 'IN_PROGRESS',
                                        'COMPLETED', 'CANCELLED')),
    deadline          DATE,
    assigned_user_id  UUID REFERENCES users (id) ON DELETE SET NULL,
    created_by        UUID REFERENCES users (id) ON DELETE SET NULL,
    archived_at       TIMESTAMPTZ,
    created_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    version           BIGINT        NOT NULL DEFAULT 0,
    CONSTRAINT customer_orders_number_uk UNIQUE (workspace_id, order_number),
    CONSTRAINT customer_orders_id_workspace_uk UNIQUE (id, workspace_id),
    -- клиент должен принадлежать тому же рабочему пространству
    CONSTRAINT customer_orders_client_fk FOREIGN KEY (client_id,
workspace_id)
        REFERENCES clients (id, workspace_id),
    -- в архив можно перенести только завершённый заказ
    CONSTRAINT customer_orders_archive_check
        CHECK (archived_at IS NULL OR status IN ('COMPLETED', 'CANCELLED'))
);

CREATE TABLE payments (
    id               UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id     UUID          NOT NULL,
    order_id         UUID          NOT NULL,
    amount           NUMERIC(14, 2) NOT NULL CHECK (amount > 0),
    payment_method   VARCHAR(15)   NOT NULL
                     CHECK (payment_method IN ('CASH', 'CARD',
                                               'BANK_TRANSFER', 'OTHER')),
    payment_date     DATE          NOT NULL,
    reference        VARCHAR(100),
    idempotency_key  UUID          NOT NULL,
    created_by       UUID REFERENCES users (id) ON DELETE SET NULL,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    voided_at        TIMESTAMPTZ,
    voided_by        UUID REFERENCES users (id) ON DELETE SET NULL,
    void_reason      VARCHAR(300),
    CONSTRAINT payments_order_fk FOREIGN KEY (order_id, workspace_id)
        REFERENCES customer_orders (id, workspace_id),
    CONSTRAINT payments_idempotency_uk UNIQUE (workspace_id,
idempotency_key),
    CONSTRAINT payments_void_check
        CHECK ((voided_at IS NULL) = (void_reason IS NULL))
);
```